

AFFERENT INFORMATION DURING THE SIMULATION

By Sergio Verduzco Flores,
Okinawa Institute of Science and Technology,
June 2016

General description of the source code

This project provides the source code for 6 C++ classes and one main program that shows how to use them. The classes are:

1) Millard12EqMuscleWithAfferents

This is a muscle type derived from Millard2012EquilibriumMuscle. It has four additions with respect to this parent class: an object of the class Mileusnic06Spindle, an object of the class Lin02GolgiTendonOrgan, a state variable corresponding to the low-pass filtered muscle fiber velocity, and a state variable corresponding to the low-pass filtered muscle fiber acceleration. The force generating dynamics of the Millard2012EquilibriumMuscle are left untouched.

Unlike most muscle types, Millard12EqWithAfferents expects 3 inputs. One input causes the muscle activation, and the other two inputs correspond to the gamma motoneuron input that modulates the response of a muscle spindle. The inputs can be specified using objects of the SpindleController, or the TugOfWarController classes.

2) Mileusnic06Spindle

This object implements the spindle afferent model from: Mileusnic et al. "Mathematical Models of Proprioceptors. I. Control and Transduction in the Muscle Spinle" J Neurophysiol 96: 1772-1788, 2006. The implementation requires 8 state variables to implement the dynamics of the model, and 2 cache variables to place the outputs (the activity of Ia and II afferents).

3) Lin02GolgiTendonOrgan

This object implements a model for the aggregate population response of the Golgi tendon organs in a muscle. The model is taken from: Lin & Crago, "Neural and Mechanical Contributions to the Stretch Reflex: A Model Synthesis" Ann Biomed Eng 30:54-67. Implementing this model requires 4 state variables, and one cache variable is used to provide the output, corresponding to the aggregate firing rate of all Golgi tendon organs connected to the muscle.

4) SpindleController

This is a concrete Controller that can be used to provide inputs to the Millard12EqMuscleWithAfferents muscle using prescribed functions. The use of this Controller is similar to using the class PrescribedController, although getting controls from files has not been implemented yet.

An example of how to use the SpindleController class is in the file mainAfferents.cpp .

5) TugOfWarController

This is another concrete Controller. It is based on the proportional-derivative controller of the “Creating a Controller” tutorial

(<http://simtk-confluence.stanford.edu:8080/display/OpenSim/Creating+a+Controller>) .

The customized controller from the tutorial is adapted by adding lines specifying the inputs to the muscle spindle:

```
Vector muscleControl(3, 0.0);
muscleControl[0] = leftControl;
muscleControl[1] = 0.01; // dynamic gamma inputs
muscleControl[2] = 0.01; // static gamma motoneuron inputs
```

The inputs to the muscle spindle are constant here, but in the context of this class it is straightforward to make them an arbitrary function of the time, or some other simulation variable.

An example of how to use the TugOfWarController class is in the mainAfferents.cpp program.

6) AfferentAnalysis

This is a custom analysis object that creates and allocates 3 storage objects to report the output of the simulation run in the mainAfferents.cpp program. One storage object stores the inputs to the muscle (including the inputs to the spindle afferent), another storage object stores the outputs of the muscle spindle, and the remaining storage output handles the output of the Golgi tendon organ.

mainAfferents.cpp

This program runs a simulation of the tug-of-war with two muscles, where one of the muscles is of the type Millard12EqMuscleWithAfferents, and the other muscle is of the type Millard2012EquilibriumMuscle. This main program is a modified version of the mainFatigue.cpp program used in the “Creating a Customized Muscle Model” tutorial (<http://simtk-confluence.stanford.edu:8080/display/OpenSim/Creating+a+Customized+Muscle+Model>). The steps taken in mainAfferents.cpp should be easy to understand to anyone who has completed the tutorial.

Unlike mainFatigue.cpp, mainAfferents.cpp has two alternatives for defining a controller, namely with a TugOfWarController, or with a SpindleController. Choosing one of these two only requires to

uncomment the code for implementing the desired controller, and commenting the code to implement the other one.

As in the “Creating a Customized Muscle Model” tutorial, there is an MuscleAnalysis object that reports the output coming from the muscles. All this output is reported in the files of the folder MuscleAnalysisResults, and have names prepended by the string “muscle”. In addition, there is also an AfferentAnalysis object that reports the output specific to Millard12EqMuscleWithAfferents. This output will be in the 3 files whose name is prepended with “afferents”.

Compiling and running the source code

The example on how to use the source code (mainAfferents.cpp) was designed so that it could be run in the same manner as the “Creating a Customized Muscle Model” tutorial. The steps to compile the source code are the same as for any extension of the OpenSim API. In short, a project (.sln) file is created with CMake, and the project is compiled with Visual Studio. This tutorial covers some of the necessary background for people unfamiliar with this process:

<http://simtk-confluence.stanford.edu:8080/display/OpenSim24/Developer%27s+Guide>

The current release of the program (v0.9) was compiled using CMake 3.5.0, and Microsoft Visual Studio Community 2015. The CMakeLists.txt file used is included with the distribution.